

Materia: Programación III

UNIDAD 1 A 4 · DEL OBJETO A LA PERSISTENCIA

TRABAJO PRÁCTICO

Librería: catálogo, clientes y ventas con FastAPI y PostgreSQL

Objetivos

El trabajo integra en un único proyecto lo desarrollado en los capítulos 1 a 4: los fundamentos de FastAPI (capítulo 1), los contratos y el CRUD (capítulo 2), la ejecución asíncrona sin bloqueos (capítulo 3) y la persistencia relacional con SQLAlchemy, PostgreSQL y Alembic, incluidas las relaciones entre entidades (capítulo 4).

Al finalizar, el proyecto entregado debe demostrar que el alumno puede:

1. Construir una API asíncrona con FastAPI, con parámetros tipados, dependencias declaradas con `Depends()` y recursos creados en el ciclo de vida de la aplicación.
2. Separar los modelos de tabla de los contratos de entrada y de salida, y responder con los códigos HTTP correctos.
3. Traducir un modelo con asociación, agregación, composición, uno a uno y muchos a muchos a un esquema relacional, justificando de qué lado va cada clave foránea.
4. Declarar la navegación en los dos extremos de cada relación y leerla siempre con carga anticipada, sin caer en el problema N+1.
5. Versionar el esquema con migraciones de Alembic, leyendo cada migración generada antes de aplicarla.
6. Delegar en la base las garantías que la base puede dar: unicidad, claves foráneas y comprobaciones.

QUÉ QUEDA AFUERA

No se evalúan autenticación ni autorización (capítulo 5), repositorios ni Unit of Work (capítulo 6), ni colas de tareas con Redis (capítulos 7 y 8). La lógica vive en el `service` de cada módulo, que recibe la sesión como parámetro.

Contexto

Una librería de barrio necesita una API para su catálogo y sus ventas. Vende libros de distintas editoriales, clasificados por género y escritos por uno o varios autores, que pueden figurar como autor, coautor, traductor o ilustrador. Registra a sus clientes, que pueden cargar un perfil con teléfono y dirección, y registra cada venta con sus renglones: qué libro, cuántos ejemplares y a qué precio.

El dominio está elegido para que aparezcan todas las relaciones vistas en el capítulo 4, cada una con una razón concreta:

Relación	Entre	Por qué es de ese tipo
Agregación	Libro y editorial	El libro incorpora una editorial que ya existe: viaja su id, y la misma editorial la comparten muchos libros
Agregación opcional	Libro y género	El libro puede incorporar un género que ya existe; es optativo: un libro puede no tenerlo
Muchos a muchos con datos	Libro y autor	Un libro tiene varios autores y un autor varios libros; el rol y el orden son de la relación
Uno a uno	Cliente y perfil	Un cliente tiene como máximo un perfil
Asociación	Venta y cliente	La venta conoce al cliente que compró, pero ninguno es parte del otro
Composición	Venta y renglón	Un renglón no tiene sentido sin su venta: se crea con ella
Asociación	Renglón y libro	El renglón nombra un libro que ya existe

Requisitos técnicos

Componente	Versión mínima	Uso
Python	3.12	Lenguaje
FastAPI	0.111	Framework web; todos los handlers con <code>async def</code>
SQLModel	0.0.24	Modelos de tabla y contratos
SQLAlchemy	2.0, asincrónico	Motor, sesión y carga anticipada
asyncpg	0.30	Controlador asincrónico de PostgreSQL
PostgreSQL	16	Base de datos

Componente	Versión mínima	Uso
Alembic	1.13, plantilla asincrónica	Migraciones
python-dotenv	1.0	Lectura del archivo .env
uvicorn	El que instala fastapi[standard]	Servidor ASGI

La dirección de la base se lee de la variable de entorno `DATABASE_URL`, definida en un archivo `.env` que no se versiona. El repositorio incluye un `env.example` con la forma del valor y sin ningún secreto (sección 4.13).

Modelo de dominio

Los nombres de tablas y columnas son obligatorios: los casos de prueba y la corrección los usan. Se pueden agregar columnas, pero no quitar ni renombrar las indicadas.

La Figura 1 muestra el modelo como diagrama de clases, con el tipo de cada relación y sus multiplicidades: el rombo lleno es composición, el rombo vacío es agregación y la flecha es asociación. Las tablas que siguen muestran el mismo modelo traducido al esquema, con sus claves y restricciones.

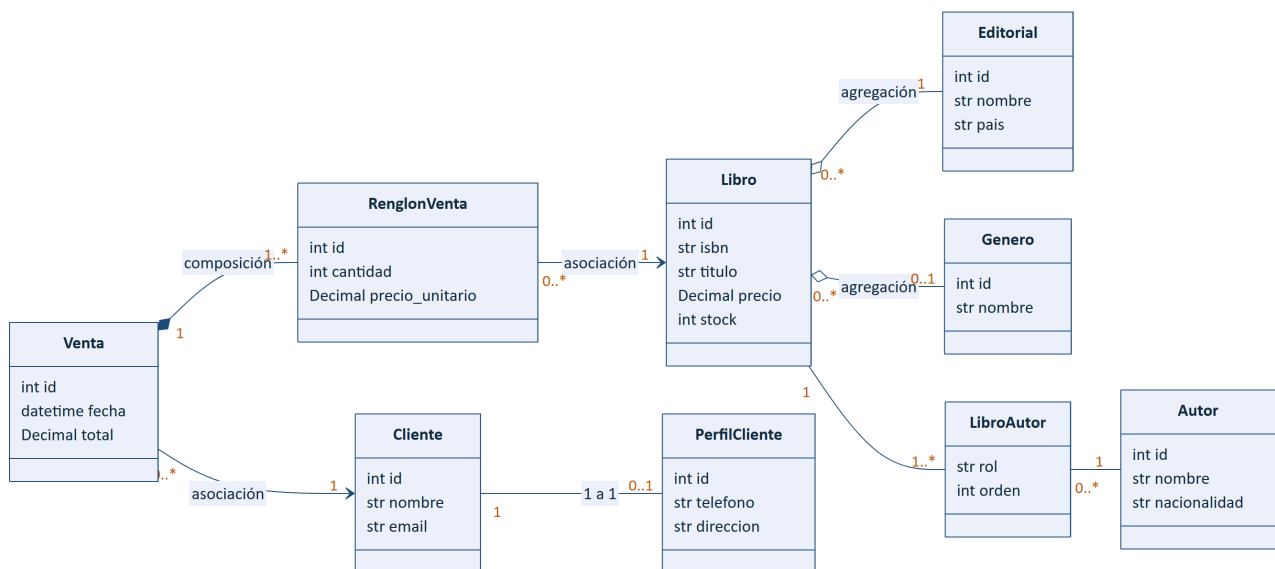


Figura 1 • Diagrama de clases. `LibroAutor` es la clase intermedia del muchos a muchos entre libro y autor, con el rol y el orden de cada autoría.

Tabla	Columnas	Restricciones
editorial	id, nombre, pais	nombre UNIQUE
genero	id, nombre	nombre UNIQUE

Tabla	Columnas	Restricciones
autor	id, nombre, nacionalidad	Sin restricciones extra
libro	id, isbn, titulo, precio (NUMERIC 10,2), stock, editorial_id, genero_id (nulo permitido)	isbn UNIQUE; CHECK precio > 0; CHECK stock ≥ 0
libro_autor	libro_id, autor_id, rol, orden	Clave primaria (libro_id, autor_id); CHECK orden ≥ 1; CHECK de rol dentro de autor, coautor, traductor, ilustrador
cliente	id, nombre, email	email UNIQUE
perfil_cliente	id, cliente_id, telefono, direccion	cliente_id UNIQUE
venta	id, cliente_id, fecha (con zona horaria), total (NUMERIC 12,2)	CHECK total ≥ 0
renglon_venta	id, venta_id, libro_id, cantidad, precio_unitario (NUMERIC 10,2)	CHECK cantidad > 0; CHECK precio_unitario > 0

Claves foráneas y navegación

Cada clave foránea lleva un índice (PostgreSQL no lo crea solo, sección 4.10) y navegación en los dos extremos con `Relationship(back_populates=...)`.

LA TABLA INTERMEDIA ES UNA ENTIDAD

`libro_autor` se modela como objeto de asociación, con su propia clase, porque tiene datos que no son ni del libro ni del autor: el rol y el orden en la tapa. Una relación muchos a muchos sin clase intermedia (`link_model`) escondería esos datos. Para ir de un libro a sus autores se navega `libro.autorias[i].autor`.

Reglas de negocio

Regla	Enunciado	Garante esperado
RN-01	El ISBN de un libro es único.	UNIQUE en la base; 409
RN-02	El email de un cliente es único.	UNIQUE en la base; 409
RN-03	El precio de un libro es mayor que cero y el stock nunca es negativo.	CHECK en la base y validación del DTO

Regla	Enunciado	Garante esperado
RN-04	Una venta se crea con sus renglones: los datos de cada renglón viajan dentro del cuerpo del alta de la venta, y no existe un alta de renglones por separado.	DTO anidado; los renglones se agregan a <code>venta.renglones</code>
RN-05	Una venta tiene al menos un renglón, y un mismo libro no aparece en dos renglones.	Validación del DTO; 422
RN-06	El precio de cada renglón se copia del libro al registrar la venta y no cambia aunque después cambie el precio del libro.	El servidor lo lee de la base; nunca viaja en el DTO
RN-07	El total de la venta lo calcula el servidor como suma de cantidad por precio unitario.	No viaja en el DTO
RN-08	Las ventas no se modifican: registran un hecho (sección 4.9).	No hay PATCH de ventas
RN-09	Un cliente tiene como máximo un perfil.	UNIQUE en <code>perfil_cliente.cliente_id</code> ; 409
RN-10	Todo listado se pagina con <code>limit</code> (de 1 a 100, por defecto 20) y <code>offset</code> (desde 0).	Restricciones en Query; 422
RN-11	El listado de libros se puede filtrar por parte del título, sin distinguir mayúsculas.	Parámetro ligado, nunca SQL armado con texto
RN-12	Ningún error devuelve al cliente texto interno de la base (nombres de tablas, restricciones o trazas).	Traducción del <code>IntegrityError</code> a un mensaje propio

Estructura obligatoria del proyecto

La organización es por módulo funcional, con la misma estructura en todos los módulos:

Ruta	Contenido
<code>app/main.py</code>	Aplicación, lifespan y registro de routers
<code>app/core/config.py</code>	Lectura de <code>DATABASE_URL</code> desde el entorno
<code>app/core/database.py</code>	Motor, fábrica de sesiones y dependencia <code>get_session</code>
<code>app/modules/<módulo>/models.py</code>	Modelos de tabla, restricciones y relaciones
<code>app/modules/<módulo>/schemas.py</code>	DTO de entrada (<code>Create</code> , <code>Update</code>) y de salida (<code>Read</code>)
<code>app/modules/<módulo>/service.py</code>	Lógica, traducción de errores y lecturas con precarga
<code>app/modules/<módulo>/router.py</code>	Rutas HTTP: reciben, delegan y declaran la respuesta

Ruta	Contenido
migrations/	Configuración de Alembic y sus versiones
test/	Archivos <code>.http</code> con los casos de prueba

Módulos mínimos: `health`, `editoriales`, `generos`, `autores`, `libros`, `clientes` y `ventas`.
`LibroAutor` vive en `libros`, `PerfilCliente` en `clientes` y `RenglonVenta` en `ventas`.

Consignas

Ejercicio 1 · Proyecto, conexión y ciclo de vida

10 puntos

Crear el proyecto con la estructura obligatoria. El motor asincrónico y la fábrica de sesiones se crean en el lifespan de la aplicación, se guardan en `app.state` y el motor se cierra al apagar (sección 1.9). La sesión llega a cada handler con `Depends(get_session)`, una por petición, creada con `expire_on_commit=False` (sección 4.5). La dirección de la base se lee de `.env`. Se expone `GET /health/live`, que no toca la base, y `GET /health/ready`, que ejecuta `SELECT 1`.

Ejercicio 2 · Modelos, restricciones y relaciones

25 puntos

Escribir los modelos de tabla del modelo de dominio con todas sus restricciones (UNIQUE, CHECK, claves foráneas e índices). Declarar la navegación en los dos extremos de cada clave foránea. `Cliente.perfil` devuelve un objeto y no una lista. El dinero se representa con `Decimal`, nunca con `float`.

Ejercicio 3 · Migraciones

10 puntos

El esquema se crea exclusivamente con Alembic (plantilla asincrónica); no se usa `create_all`. Se entregan al menos dos migraciones generadas con `--autogenerate`: la primera con todas las tablas menos `genero`, y la segunda que agrega la tabla `genero` y la columna `libro.genero_id`. Cada migración se lee antes de aplicarla: si la versión generada tiene un problema, se corrige a mano y se explica con un comentario en el propio archivo. `alembic upgrade head` sobre una base vacía y `alembic downgrade -1` seguido de `upgrade head` deben funcionar.

Ejercicio 4 · Contratos y CRUD

15 puntos

Implementar los endpoints de referencia con DTO separados de los modelos de tabla. Los DTO de alta no llevan `id`. Los PATCH aplican sólo los campos enviados (`exclude_unset`). Los códigos de respuesta son 201 al crear, 404 si el recurso no existe, 409 si la base rechaza la escritura y 422 si el cuerpo o los parámetros no cumplen el contrato. Los listados se paganinan (RN-10) y la búsqueda por título usa parámetros ligados (RN-11).

Ejercicio 5 · Altas con relaciones: ids y datos

20 puntos

El alta de un libro recibe los ids de cosas que ya existen: `editorial_id`, `genero_id` y la lista `autores`, donde cada elemento trae `autor_id` más los datos de la relación (`rol` y `orden`). El alta de una venta recibe `cliente_id` (asociación: viaja un id) y la lista `renglones` con `libro_id` y `cantidad` (composición: viajan los datos de la parte, que nace con la venta). La venta se crea con sus renglones y cumple RN-04 a RN-07: el precio de cada renglón se copia del libro y el total lo calcula el servidor. En el alta del libro ningún `if` consulta antes si un id existe: se escribe, y si la base rechaza la clave foránea, se responde 409.

Ejercicio 6 · Lecturas sin N+1

10 puntos

Toda respuesta que incluya datos de una relación los obtiene con carga anticipada declarada en la misma consulta: `joinedload` para relaciones a uno y `selectinload` para colecciones (sección 4.6). El detalle de un libro (con editorial, género y autores) y el detalle de una venta (con cliente y renglones) se resuelven con un número fijo de consultas, que no crece con la cantidad de autores o de renglones. Ninguna lectura puede depender de la carga perezosa.

Ejercicio 7 · Asincronía sin bloqueos

10 puntos

Todos los handlers son `async def` y ninguno ejecuta una llamada bloqueante (`time.sleep`, `requests`, acceso síncrono a la base). Al registrar una venta se agenda con `BackgroundTasks` el aviso del comprobante al cliente, simulado con `await asyncio.sleep(0.5)` y un mensaje en el log, que corre después de enviar la respuesta. Mientras corre el aviso, el servidor sigue atendiendo otras peticiones.

Endpoints de referencia

Método	Ruta	Cuerpo	Respuesta	Códigos
GET	/health/live	Sin cuerpo	{"estado": "vivo"}	200
GET	/health/ready	Sin cuerpo	Estado de la base	200
POST	/editoriales/	EditorialCreate	EditorialRead	201, 409, 422
GET	/editoriales/	limit, offset	Lista de EditorialRead	200, 422
GET	/editoriales/{id}	Sin cuerpo	EditorialRead	200, 404, 422
PATCH	/editoriales/{id}	EditorialUpdate	EditorialRead	200, 404, 409
GET	/editoriales/{id}/libros	Sin cuerpo	Libros de la editorial	200, 404
POST	/generos/	GeneroCreate	GeneroRead	201, 409
GET	/generos/	Sin cuerpo	Lista de GeneroRead	200
GET	/generos/{id}/libros	Sin cuerpo	Libros del género	200, 404
POST	/autores/	AutorCreate	AutorRead	201, 422
GET, PATCH	/autores/{id}	AutorUpdate en PATCH	AutorRead	200, 404, 422
GET	/autores/{id}/libros	Sin cuerpo	Libros con rol y orden	200, 404
POST	/libros/	LibroCreate con autores	Detalle del libro	201, 409, 422
GET	/libros/	limit, offset, titulo, genero_id	Libros con editorial y género	200, 422
GET	/libros/{id}	Sin cuerpo	Libro con editorial, género y autores	200, 404
PATCH	/libros/{id}	LibroUpdate	Detalle del libro	200, 404, 409, 422
POST	/clientes/	ClienteCreate	ClienteRead	201, 409, 422

Método	Ruta	Cuerpo	Respuesta	Códigos
GET	/clientes/{id}	Sin cuerpo	Cliente con su perfil	200, 404
PATCH	/clientes/{id}	ClienteUpdate	Cliente con su perfil	200, 404, 409, 422
POST	/clientes/{id}/perfil	PerfilCreate	PerfilRead	201, 404, 409
GET	/clientes/{id}/ventas	Sin cuerpo	Ventas del cliente	200, 404
POST	/ventas/	VentaCreate con renglones	Venta con cliente y renglones	201, 409, 422
GET	/ventas/	limit, offset	Lista de ventas	200, 422
GET	/ventas/{id}	Sin cuerpo	Venta con cliente y renglones	200, 404

Ejemplo del cuerpo de POST /ventas/. Viaja el id del cliente y los datos de cada renglón; no viajan el precio, el total ni la fecha:

```
{
  "cliente_id": 2,
  "renglones": [
    { "libro_id": 1, "cantidad": 2 },
    { "libro_id": 4, "cantidad": 1 }
  ]
}
```

Respuesta esperada, con el precio copiado de cada libro y el total calculado por el servidor:

```
{
  "id": 2,
  "fecha": "2026-10-08T21:14:11.751581Z",
  "total": "77000.00",
  "cliente": { "id": 2, "nombre": "Tomás Ríos", "email": "tomas@example.com" },
  "renglones": [
    { "libro_id": 1, "titulo": "Rayuela", "cantidad": 2, "precio_unitario": "25000.00", "subtotal": "50000.00" },
    { "libro_id": 4, "titulo": "Respiración artificial", "cantidad": 1, "precio_unitario": "27000.00", "subtotal": "27000.00" }
  ]
}
```

Ejemplo del cuerpo de POST /libros/. Viajan ids de editorial, género y autores, más los datos propios de cada autoría:

```
{
  "isbn": "978-950-00-0006-6",
  "titulo": "Los que aman, odian",
  "precio": "21000.00",
  "stock": 4,
  "editorial_id": 1,
  "genero_id": 1,
  "autores": [
    { "autor_id": 3, "rol": "coautor", "orden": 1 },
    { "autor_id": 4, "rol": "coautor", "orden": 2 }
  ]
}
```

Requisitos no funcionales

Requisito	Detalle
Asincronía	Todos los handlers con <code>async def</code> ; ninguna llamada bloqueante dentro de ellos
Credenciales	Ninguna contraseña en el código, en <code>alembic.ini</code> ni en el repositorio
Carga perezosa	Prohibida: toda relación que se lee se precarga en la consulta
SQL	Nunca armado por concatenación de texto con datos del usuario
Errores	Mensajes propios; nunca el texto de la base ni una traza
Unicidad	Garantizada por la base; no se verifica con una consulta previa
Diagnóstico	Las consultas emitidas se pueden ver en la consola (por ejemplo, <code>echo=True</code> configurable)

Casos de prueba mínimos

El proyecto incluye un script o comando que deja la base en un estado inicial conocido, y archivos `.http` en `test/` que demuestran, como mínimo, los siguientes casos. Cada caso indica la respuesta esperada en un comentario.

Caso	Qué se demuestra	Resultado esperado
C-01	Un parámetro de ruta que no es entero	422, sin ejecutar el handler
C-02	ISBN o email repetido	409 con mensaje propio
C-03	Segundo perfil para el mismo cliente	409

Caso	Qué se demuestra	Resultado esperado
C-04	Libro con una editorial o un autor inexistente	409 con mensaje propio
C-05	Venta con un libro o un cliente inexistente	409
C-06	Venta válida con dos renglones	201, con los renglones creados, los precios copiados y el total calculado
C-07	Detalle de un libro y de una venta	Cantidad fija de consultas en la consola, la misma con uno o con varios autores o renglones
C-08	Búsqueda por título con el texto ' OR '1'='1	200 con lista vacía

Criterios de evaluación

Criterio	Peso	Qué se mira
Proyecto, conexión y ciclo de vida	10 %	Estructura, lifespan, <code>Depends</code> , <code>.env</code> y <code>env.example</code>
Modelos, restricciones y relaciones	25 %	Lado de cada clave foránea, uno a uno con UNIQUE, objeto de asociación, CHECK, índices, navegación en los dos extremos
Migraciones	10 %	Dos migraciones autogeneradas, leídas y corregidas si hacía falta; upgrade y downgrade funcionando
Contratos y CRUD	15 %	DTO separados, códigos HTTP, PATCH parcial, paginación, parámetros ligados
Altas con relaciones	20 %	Ids en agregación y asociación, datos de los renglones dentro de la venta en la composición
Lecturas sin N+1	10 %	Carga anticipada adecuada a cada relación; número fijo de consultas
Asincronía sin bloqueos	10 %	Handlers asíncronos sin llamadas bloqueantes; aviso en segundo plano con <code>BackgroundTasks</code>

Un proyecto que no levanta, que no crea su esquema con `alembic upgrade head` sobre una base vacía o que contiene credenciales reales en el repositorio no se corrige hasta que se subsane.

Formato de entrega

- Repositorio con el código fuente completo, `requirements.txt`, `env.example`, la carpeta `migrations/` y la carpeta `test/`. El archivo `.env` no se sube.

-
- `README.md` con los pasos para levantar el proyecto desde cero: entorno virtual, dependencias, `.env`, `alembic upgrade head`, carga de datos iniciales y arranque del servidor.
 - Evidencia del caso C-07: una captura o un fragmento de la consola con las consultas del detalle de un libro y de una venta.